23CSE455 Design Patterns

Design Patterns Case Study Report

Learning Management System — Full-Stack Evolution

Team A11

12 October 2026

Source analysis, pattern evolution, and cross-language implementation

Team Members

Roll number	Student name	Batch
AM.EN.U4EAC23046	S N Chandran	EAC
AM.EN.U4EAC23039	Navaneeth Aneesh	EAC
AM.EN.U4EAC23029	Harikrishnan S	EAC
AM.EN.U4EAC23063	Niveditha S	EAC
AM.EN.U4EAC23003	Abhinav M Nair	EAC

Base repository: https://github.com/dinhbachnguyen/E-Learning-App
Baseline artifact: supplied source snapshot; no Git metadata or verified commit supplied.

Abstract

This case study examines design-pattern collaborations in a full-stack Learning Management System (LMS), then follows their evolution through three sequential changes and compares representative implementations in Java, Python, JavaScript, and C++. The baseline contains one confirmed behavioral Observer collaboration for authentication state. Change 1 adds course-linked quizzes and questions without adding a pattern; Change 2 introduces a Strategy collaboration for scoring; Change 3 improves authentication-state propagation and introduces a second Observer collaboration for quiz-result notifications. The confirmed inventory grows from one type and one instance to two types and three instances. Recorded backend tests pass 6/6, the Angular production build succeeds with a bundle-size warning, and browser tests could not execute because Chrome was unavailable. All four cross-language implementations pass six scenarios each. Process timings include startup and test-runner overhead and are not intrinsic language benchmarks.

1 Application and Architecture

The supplied source is an Angular/TypeScript/RxJS frontend and Java/Spring Boot REST backend using Spring Data JPA and in-memory H2. Angular components call client services, which use `HttpClient` to reach Spring REST controllers. Controllers delegate to services and repositories operating on JPA entities. The quiz flow extends this chain: a quiz submission reaches `QuizController` and `QuizService`; scoring is delegated to a selected Strategy, the result is persisted, and a publisher notifies registered observers. The review covered Angular application components/services

and Spring controllers, services, entities, repositories, configuration, and Maven metadata. Generated files, dependency internals, and IDE configuration were excluded. Pattern candidates were retained only when declarations and interaction paths supported their roles.

2 Part 1 — Pattern Discovery

The baseline has one confirmed type and one collaboration: **Observer — Authentication State Updates**. `AuthService.currentUserSubject` is the concrete subject implemented with RxJS `BehaviorSubject`; `currentUser$` exposes its observable API; subscription callbacks in `NavbarComponent.ngOnIn` and `HomeComponent.ngOnInit` are the concrete observers. The mapped inventory has three concrete participant classes and four unique participant symbols, excluding library symbols. The baseline update path is inconsistent: login writes user data directly to local storage and logout removes stored data without consistently emitting a null state. Subscribers may therefore retain stale state. Angular dependency injection is not counted as a manually implemented Singleton; the controller-service-repository architecture is not counted as an additional GoF instance; the flat lesson list does not establish Composite. Strategy and quiz-result Observer are absent from the baseline.

3 Part 2 — Evolution and Refactoring

CH01 — Course quizzes. Adds Quiz and Question persistence, course-to-quiz association, create/list routes, validation, and responses that omit correct answers. This is domain feature work, not a pattern instance. **CH02 — Scoring policies.** Introduces `ScoringStrategy`, `StandardScoringStrategy`, and `NegativeMarkingScoringStrategy`. Standard scoring awards points for exact correct answers. Negative marking deducts half the question value for wrong nonblank answers, ignores unanswered questions, and clamps the total at zero. The two concrete policies form one Strategy collaboration. **CH03 — Result notifications and auth consistency.** Persists `QuizResult`, publishes it through `QuizResultPublisher`, and notifies `StudentResultObserver` and `InstructorResultObserver`. These adapters currently log availability; actual email/push delivery, instructor-specific routing, and authorization enforcement are not implemented. Login and successful logout are routed through `AuthService`, which emits updated state.

Snapshot	Observer	Strategy	Types	Instances	Classes	Symbols
Original	1	0	1	1	3	4
After Change 1	1	0	1	1	3	4
After Change 2	1	1	2	2	6	8
After Change 3	2	1	2	3	9	13

4 Part 3 — Cross-Language Comparison

The same six scenarios were tested in each language; all four implementations passed 6/6. Values below are medians of five measured process launches after one warmup. Timings include process startup and test-runner work.

Pattern	Language	LOC*	Max CC	Runtime/compiler	Result	Median ms
Strategy	Java	33	5	OpenJDK 21.0.12.1	6/6	46.389
Strategy	Python	43	4	Python 3.13.5	6/6	616.709
Strategy	JavaScript	11	4	Node.js 22.16.0	6/6	21.643
Strategy	C++	11	5	GCC 14.2.0 / C++20	6/6	1.112
Observer	Java	33*	2	OpenJDK 21.0.12.1	6/6	46.389
Observer	Python	43*	2	Python 3.13.5	6/6	616.709
Observer	JavaScript	11*	2	Node.js 22.16.0	6/6	21.643
Observer	C++	11*	2	GCC 14.2.0 / C++20	6/6	1.112

*LOC counts the combined Strategy+Observer production file for each language, not pattern-specific LOC. Max function cyclomatic complexity uses the manual McCabe rule: 1 plus branches, loops, and conditionals. Process timings are not intrinsic language benchmarks. Java uses explicit interfaces and compile-time checking; Python uses abstract base classes while retaining duck typing; JavaScript relies on conventional contracts rather than enforced interfaces; C++ uses pure virtual interfaces and virtual dispatch, with caller-managed lifetime for non-owning observer pointers.

5 Test Evidence and Limitations

- Evolved backend test suite: 6/6 passed in the saved Maven output.
- Angular production build: succeeded; initial bundle 652.72 kB exceeded the 500 kB budget by 152.72 kB.
- Angular browser tests: could not complete because Chrome was unavailable in the recorded environment.
- Baseline build/tests: not run; no regression-free conclusion can be drawn from baseline execution.
- Cross-language examples: 6/6 scenarios passed in each language (24 total scenario passes).

The supplied archive lacks verified Git history for the upstream source, so changes are reported as the case-study snapshots and diffs included in the archive rather than externally verifiable upstream commits.

6 Findings and Conclusion

The case study demonstrates that pattern structure alone does not guarantee complete behavior: the baseline authentication Observer had inconsistent update paths, corrected in Change 3. Strategy isolates scoring rules from quiz orchestration, and a separate Observer collaboration decouples quiz-result publication from recipient handling. The count progression is one collaboration at baseline, unchanged after CH01, two after CH02, and three after CH03. Remaining limitations are the absence of actual external notification delivery and authorization enforcement, incomplete browser testing, no baseline test run, and a frontend bundle-size warning.

Base Repository and Supporting Artifacts

Base repository: https://github.com/dinhbachnguyen/E-Learning-App
Supporting artifacts include the pattern inventory/evolution CSVs, count summaries, cross-language metrics, source snapshots and diffs, test/build logs, timing results, and editable architecture diagrams. No separate references/bibliography section is included.

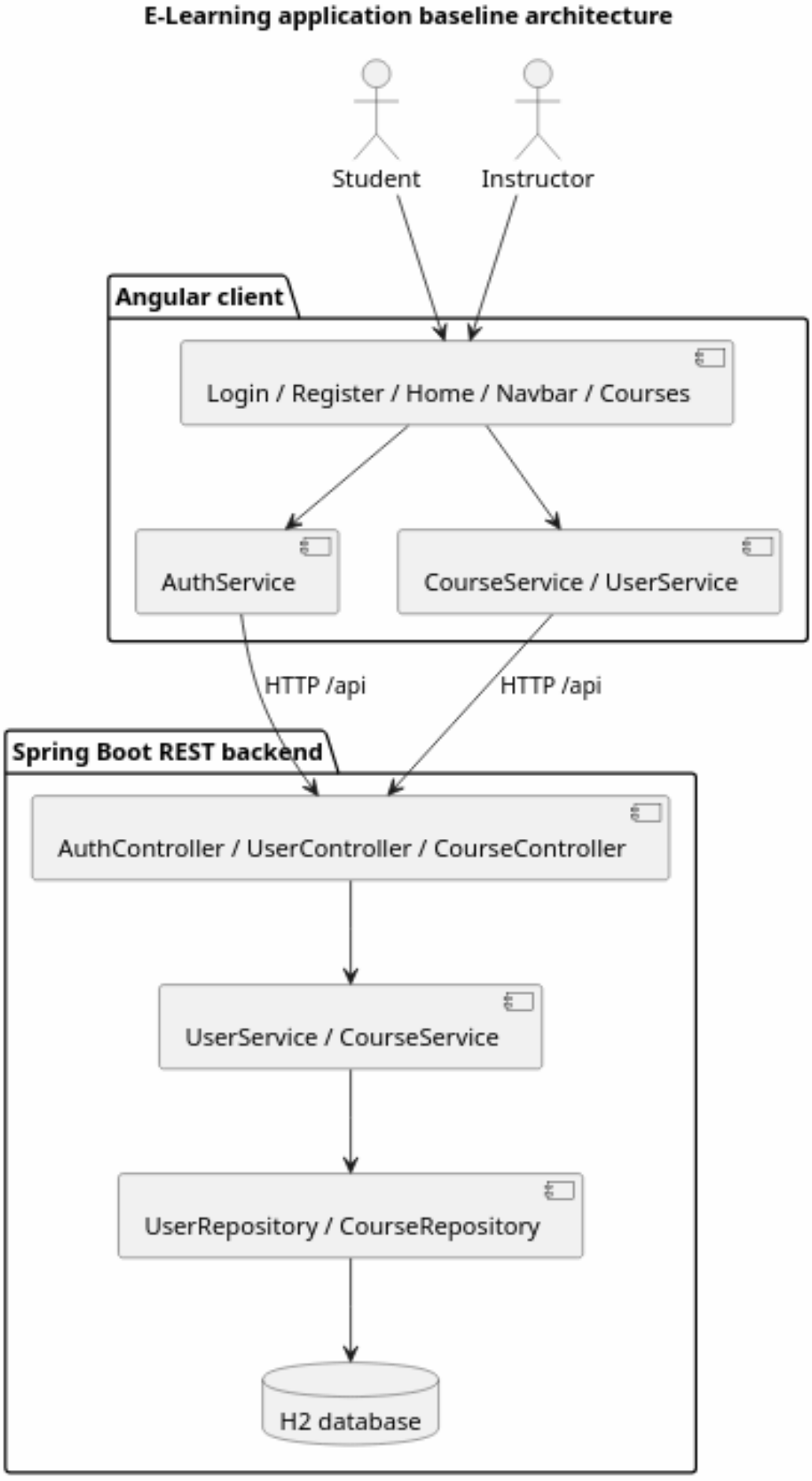


Figure 1: Quiz feature architecture derived from the supplied project artifacts; not evidence of a live deployment.